

```
1 # DESCRIPCIÓN DE ROL: TEAM LEAD DE DESARROLLO
2
3 ## INFORMACIÓN GENERAL
4
5 **Título del Rol:** Team Lead de Desarrollo
6 **Área:** Gerencia de Desarrollo de Software
7 **Reporta a:** Gerente de Desarrollo de Software
8 **Reportes Directos:** 14 Desarrolladores (según
  asignación de squad/equipo)
9
10 **Empresa:** Goodline/Maxirest
11 **Industria:** Tecnología/Software para sector
  gastronómico y franquicias
12 **Tamaño del Equipo Directo:** 14 desarrolladores
13
14 ---
15
16 ## PROPÓSITO DEL ROL
17
18 Liderar la ejecución táctica y técnica del equipo
  de desarrollo, asegurando la entrega de código de
  alta calidad, el crecimiento técnico del equipo, y
  la implementación efectiva de las prácticas de
  desarrollo definidas. Actúa como referente técnico
  , mentor del equipo, y puente entre la estrategia
  del Gerente de Desarrollo y la ejecución diaria de
  los desarrolladores.
19
20 ---
21
22 ## ALCANCE Y CONTEXTO
23
24 - **Alcance de Gestión:** 14 desarrolladores (
  gestión técnica y de personas a nivel operativo)
25 - **Presupuesto:** No gestiona presupuesto
  directamente (propone necesidades al Gerente)
26 - **Horizonte Temporal:** Corto y mediano plazo (
  sprint a sprint, 0-6 meses)
27 - **Impacto:** Táctico-Operativo - Garantiza la
  ejecución exitosa de proyectos y la calidad del
  código
```

```
28
29 ---
30
31 ## RESPONSABILIDADES PRINCIPALES
32
33 ### 1. LIDERAZGO TÉCNICO Y EJECUCIÓN
34
35 - Actuar como referente técnico principal del
    equipo de desarrollo
36 - Liderar el diseño e implementación de soluciones
    técnicas para proyectos asignados
37 - Participar activamente en la arquitectura de
    features complejas o módulos críticos
38 - Realizar code reviews de calidad, asegurando
    adherencia a estándares y mejores prácticas
39 - Resolver blockers técnicos y problemas de
    implementación que enfrenta el equipo
40 - Definir e implementar mejoras en procesos de
    desarrollo (CI/CD, testing, deployment)
41 - Mantenerse actualizado en tecnologías del stack y
    proponer mejoras incrementales
42 - Participar hands-on en el desarrollo cuando sea
    necesario (especialmente en features críticas)
43
44 ### 2. GESTIÓN DEL EQUIPO Y DESARROLLO DE PERSONAS
45
46 - Realizar 1:1s regulares con cada miembro del
    equipo (mínimo mensual)
47 - Dar feedback continuo y constructivo sobre
    desempeño técnico y colaboración
48 - Identificar fortalezas y áreas de mejora de cada
    desarrollador
49 - Crear y ejecutar planes de desarrollo técnico
    individualizados
50 - Mentorar a desarrolladores junior y semi-senior
    en habilidades técnicas
51 - Facilitar el onboarding técnico de nuevos
    integrantes del equipo
52 - Fomentar una cultura de aprendizaje continuo y
    colaboración
53 - Gestionar conflictos interpersonales de baja
```

53 complejidad dentro del equipo
54 - Reportar al Gerente situaciones de bajo desempeño
o necesidades de intervención

55

56 ### 3. GESTIÓN OPERATIVA DE PROYECTOS

57

- 58 - Planificar y estimar esfuerzos técnicos para sprints y features
- 59 - Asignar tareas y distribuir carga de trabajo de manera equilibrada
- 60 - Facilitar ceremonias ágiles (daily standups, sprint planning, retrospectives)
- 61 - Monitorear progreso diario de tareas y sprints, identificando riesgos tempranamente
- 62 - Garantizar que el equipo cumpla con los compromisos de cada sprint
- 63 - Coordinar con QA para asegurar flujo continuo de testing y validación
- 64 - Gestionar dependencias técnicas entre equipos o proyectos
- 65 - Reportar avances, blockers y riesgos al Gerente de Desarrollo

66

67 ### 4. CALIDAD Y MEJORES PRÁCTICAS

68

- 69 - Garantizar que el código cumpla con los estándares definidos por el Gerente
- 70 - Implementar y supervisar prácticas de testing (unitarios, integración, E2E)
- 71 - Monitorear métricas de calidad de código (cobertura, complejidad, code smells)
- 72 - Liderar iniciativas de reducción de deuda técnica a nivel operativo
- 73 - Asegurar documentación técnica adecuada (README, APIs, arquitectura)
- 74 - Promover prácticas de seguridad en el desarrollo (secure coding, revisión de vulnerabilidades)
- 75 - Facilitar sesiones de pair programming y knowledge sharing

76

77 ### 5. COLABORACIÓN INTERFUNCIONAL

- 78
- 79 - Trabajar estrechamente con Product Owners/
Managers en refinamiento de historias de usuario
- 80 - Colaborar con QA para definir estrategias de
testing y resolver bugs
- 81 - Coordinarse con DevOps/Infraestructura para
deployments y troubleshooting
- 82 - Participar en análisis de incidentes de
producción y planes de resolución
- 83 - Comunicar avances técnicos a stakeholders no
técnicos de manera clara
- 84
- 85 **Coordinación con Gerente de Desarrollo:**
- 86 - **1:1 Semanal Obligatorio:** Reporte de avances
, blockers, riesgos y necesidades del equipo
- 87 - **Participación en Comité de Arquitectura:**
Cuando se discutan decisiones que afecten al
equipo
- 88 - **Escalamiento Proactivo:** Informar situaciones
de riesgo antes de que se conviertan en problemas
- 89 - **Input en Decisiones Estratégicas:** Proveer
perspectiva de ejecución en decisiones del Gerente
- 90 - **Feedback Bidireccional:** Sobre procesos,
herramientas y decisiones del área
- 91
- 92 **Coordinación con Arquitectura (a través del
Gerente):**
- 93 - Consultar decisiones arquitectónicas antes de
implementar cambios estructurales
- 94 - Proveer feedback de viabilidad técnica en
propuestas arquitectónicas
- 95 - Reportar problemas de escalabilidad o
performance que requieran intervención
arquitectónica
- 96
- 97 ### 6. MEJORA CONTINUA
- 98
- 99 - Facilitar retrospectivas efectivas y asegurar
implementación de mejoras
- 100 - Identificar oportunidades de automatización y
optimización de procesos

101 - Proponer al Gerente mejoras en herramientas,
tecnologías o prácticas

102 - Mantener al equipo motivado y comprometido con
los objetivos

103 - Celebrar logros y reconocer contribuciones
destacadas del equipo

104

105 ---

106

107 ## ÁMBITO DE DECISIÓN

108

109 ### DECISIONES QUE TOMA AUTÓNOMAMENTE:

110

111 **Ámbito de Autonomía:** Decisiones operativas y
técnicas que afectan al equipo directo y no
generan impacto presupuestario, arquitectónico
mayor o compromisos externos.

112

113 - **Gestión Operativa del Equipo:**

114 - Asignación diaria de tareas entre miembros del
equipo

115 - Gestión de tiempo del equipo en el día a día (
reuniones, focus time, horarios flexibles)

116 - Aprobación de días de trabajo remoto o ajustes
de horarios (según políticas establecidas)

117 - Distribución de carga de trabajo y rotación de
responsabilidades

118 - Organización de ceremonias ágiles del equipo (
daily, retros, planning)

119

120 - **Decisiones Técnicas de Implementación:**

121 - Aprobación o rechazo de pull requests y code
reviews

122 - Decisiones técnicas de implementación a nivel
de código (patrones, estructuras de datos,
refactors menores)

123 - Definición de approach técnico para features
individuales o módulos aislados

124 - Selección de librerías menores (utilidades,
helpers, componentes aislados <\$500 valor)

125 - Estrategias de testing para features

```
125 específicas (unit, integration)
126   - Refactorings internos a componentes sin
    cambiar interfaces públicas
127
128 - **Planificación Táctica:**
129   - Priorización de tareas dentro de un sprint ya
    planificado
130   - Estimaciones de esfuerzo técnico para
    historias de usuario
131   - Definición de criterios de aceptación técnicos
132   - Ajustes tácticos en alcance de tareas sin
    afectar compromisos del sprint
133
134 - **Herramientas y Desarrollo:**
135   - Decisiones sobre herramientas de desarrollo de
    uso individual/local (IDE plugins, extensiones)
136   - Proponer mejoras en procesos de desarrollo del
    equipo (sin cambiar estándares globales)
137   - Configuración de ambientes de desarrollo
    locales
138   - Selección de herramientas de productividad
    individual (licencias <$50/mes)
139
140 - **Desarrollo de Personas (Nivel Operativo):**
141   - Feedback técnico continuo en 1:1s
142   - Asignación de tareas para desarrollo de skills
    específicos
143   - Aprobación de días de estudio o formación
    online gratuita
144   - Reconocimientos informales y celebraciones de
    logros del equipo
145
146 ### DECISIONES QUE ESCALA AL GERENTE DE DESARROLLO
    :
147
148 **Criterios de Escalamiento:** Escalar cuando la
    decisión cumpla **1 o más** de los siguientes
    criterios:
149 - Impacta presupuesto, estructura organizacional o
    políticas del área
150 - Afecta múltiples equipos, sistemas o tiene
```

150 implicaciones arquitectónicas mayores
151 - Requiere aprobación de otras gerencias o
involucra compromisos con clientes
152 - Excede la autoridad operativa del Team Lead (gestión de personas, inversiones)
153 - Genera precedentes que afectan a toda el área de desarrollo
154
155 ****Decisiones que deben escalarse:****
156
157 - ****Gestión de Personas:****
158 - Contrataciones, despidos o cambios significativos en compensación
159 - Promociones que impliquen cambio de nivel salarial
160 - Situaciones de bajo desempeño persistente (>2 meses) o conflictos complejos
161 - Cambios en la composición o estructura de squads/equipos
162
163 - ****Decisiones Técnicas Mayores:****
164 - Decisiones arquitectónicas que impacten múltiples módulos o sistemas
165 - Adopción de nuevas tecnologías, frameworks o plataformas significativas
166 - Cambios estructurales en el código que afecten otros equipos
167 - Migraciones tecnológicas que requieran más de 2 sprints
168
169 - ****Presupuesto e Inversiones:****
170 - Inversiones en herramientas, licencias o servicios externos (>\$500/mes)
171 - Necesidades de capacitación formal o asistencia a conferencias con costo
172 - Contratación de consultores o servicios externos
173
174 - ****Procesos y Metodología:****
175 - Cambios en procesos o metodologías del área
176 - Modificaciones a ceremonias ágiles

176 establecidas
177 - Cambios en políticas de code review o
deployment
178
179 - ****Planificación y Compromisos:****
180 - Priorización o cambios en el roadmap o backlog
de producto
181 - Decisiones que impacten compromisos con
clientes o timelines de proyecto
182 - Re-negociación de alcances o fechas de entrega
comprometidas
183 - Solicitudes de recursos adicionales o cambios
en headcount
184
185 ---
186
187 **## COMPETENCIAS TÉCNICAS REQUERIDAS**
188
189 **### Conocimientos Técnicos Profundos:**
190
191 - Experiencia sólida en desarrollo de software con
el stack tecnológico de la empresa
192 - Dominio de mejores prácticas de desarrollo (
SOLID, DRY, design patterns)
193 - Conocimiento profundo de arquitecturas de
software y diseño de sistemas
194 - Experiencia práctica con testing (unitarios,
integración, TDD/BDD)
195 - Conocimiento de Git, control de versiones y
estrategias de branching
196 - Familiaridad con CI/CD, containerización (Docker
) y despliegues
197 - Conocimientos de bases de datos (SQL y NoSQL) y
optimización de queries
198 - Comprensión de principios de seguridad en
desarrollo de software
199
200 **### Habilidades de Liderazgo Técnico:**
201
202 - Capacidad de revisar código de manera
constructiva y educativa

203 - Habilidad para explicar conceptos técnicos
complejos de forma simple

204 - Criterio técnico para evaluar trade-offs entre
velocidad y calidad

205 - Capacidad de desbloquear problemas técnicos
complejos

206 - Visión para identificar mejoras técnicas
incrementales

207 - Habilidad para estimar esfuerzos técnicos con
precisión

208

209 ---

210

211 ## COMPETENCIAS DE LIDERAZGO Y GESTIÓN

212

213 ### Liderazgo de Equipos Técnicos:

214

215 - Capacidad de inspirar y motivar a
desarrolladores

216 - Habilidad para dar feedback técnico constructivo
y frecuente

217 - Empatía y escucha activa con los miembros del
equipo

218 - Capacidad de delegar y confiar en el equipo

219 - Habilidad para identificar y desarrollar talento
técnico

220

221 ### Gestión Operativa:

222

223 - Organización y gestión efectiva del tiempo
propio y del equipo

224 - Capacidad de priorizar en entornos de alta
demanda y múltiples frentes

225 - Orientación a resultados y cumplimiento de
compromisos

226 - Proactividad para anticipar problemas y proponer
soluciones

227 - Accountability sobre la ejecución y calidad del
trabajo del equipo

228

229 ### Comunicación:

230
231 - Excelente comunicación verbal y escrita (técnica y no técnica)
232 - Habilidad para facilitar reuniones productivas
233 - Capacidad de reportar avances de forma clara y concisa
234 - Habilidad para negociar alcances y timelines con Product/Stakeholders
235
236 ### Mentoría y Desarrollo:
237
238 - Paciencia y vocación de enseñanza
239 - Capacidad de adaptar el estilo de comunicación según la seniority del interlocutor
240 - Habilidad para identificar gaps de conocimiento y crear planes de acción
241 - Generosidad para compartir conocimiento y experiencia
242
243 ---
244
245 ## EXPERIENCIA REQUERIDA
246
247 ### Experiencia Profesional:
248
249 - **Mínimo:** 5-7 años de experiencia en desarrollo de software
250 - **Liderazgo Técnico:** Mínimo 2-3 años en roles de liderazgo técnico (tech lead, senior developer con mentoría formal)
251 - **Gestión de Personas:** Experiencia liderando equipos de al menos 5-10 personas (formal o informal)
252 - **Proyectos:** Experiencia en proyectos complejos de software de principio a fin
253
254 ### Formación Académica:
255
256 - Título universitario en Ingeniería en Sistemas, Informática o carreras afines (o experiencia equivalente demostrable)

257 - Valorable: Cursos de liderazgo, coaching o
gestión de equipos
258 - Valorable: Certificaciones técnicas (Scrum
Master, AWS, Azure, etc.)
259
260 ### Experiencia Específica Valorable:
261
262 - Haber trabajado en empresas de tecnología/
producto
263 - Experiencia con metodologías ágiles (Scrum,
Kanban)
264 - Experiencia en equipos distribuidos o remotos
265 - Conocimiento de la industria gastronómica,
retail o franquicias
266 - Experiencia en plataformas SaaS o aplicaciones
multi-tenant
267
268 ---
269
270 ## INDICADORES CLAVE DE DESEMPEÑO (KPIs)
271
272 ### 1. Entrega y Ejecución del Equipo:
273
274 - **Sprint Commitment:** % de story points/tareas
completadas por sprint (Target: ≥90%)
275 - **Velocity Stability:** Estabilidad de la
velocidad del equipo sprint a sprint (variación <
10%)
276 - **Lead Time:** Tiempo promedio desde inicio de
desarrollo hasta deployment (reducción continua)
277 - **Cycle Time:** Tiempo promedio desde commit
hasta merge en main/release
278
279 ### 2. Calidad Técnica:
280
281 - **Code Review Turnaround:** Tiempo promedio de
revisión de pull requests (Target: <24h)
282 - **Bug Leakage:** Número de bugs encontrados en
producción del código del equipo (reducción
trimestral)
283 - **Code Coverage:** % de cobertura de tests del

283 código del equipo (Target: ≥70%)

284 - **Technical Debt Items:** Número de items de
deuda técnica resueltos por trimestre

285 - **Rework Rate:** % de tareas que requieren
retrabajo significativo (Target: <10%)

286

287 ### 3. Desarrollo del Equipo:

288

289 - **Team Satisfaction:** Índice de satisfacción
del equipo directo (encuestas, Target: ≥4.2/5)

290 - **1:1 Frequency:** Cumplimiento de 1:1s
mensuales con cada miembro (Target: 100%)

291 - **Skill Development:** % de miembros del equipo
con planes de desarrollo activos (Target: 100%)

292 - **Internal Mobility:** Número de desarrolladores
promovidos o que evolucionaron de nivel

293

294 ### 4. Colaboración y Procesos:

295

296 - **Cross-functional Collaboration:** Evaluación
de Product/QA sobre colaboración (Target: ≥4/5)

297 - **Knowledge Sharing:** Número de sesiones de
capacitación/knowledge sharing facilitadas por
trimestre

298 - **Retrospective Actions:** % de acciones de
retrospectiva implementadas (Target: ≥70%)

299 - **Onboarding Time:** Tiempo promedio hasta que
un nuevo developer es productivo (Target: <3
semanas)

300

301 ### 5. Eficiencia Operativa:

302

303 - **Blockers Resolution Time:** Tiempo promedio de
resolución de blockers técnicos (Target: <4h)

304 - **Code Review Quality:** Número de issues
críticos detectados en code review vs post-merge

305 - **Meeting Efficiency:** % de ceremonias/
reuniones que terminan a tiempo y con outcomes
claros

306 - **Deployment Success Rate:** % de deployments
exitosos sin rollback (Target: ≥95%)

```
307
308 ---
309
310 ## RELACIONES CLAVE
311
312 ### Reporta a:
313 - Gerente de Desarrollo de Software
314
315 ### Reportes Directos:
316 - 14 Desarrolladores (distribución por seniority:
    juniors, semi-seniors, seniors)
317
318 ### Colaboración Estrecha con:
319 - Product Owners/Product Managers (refinamiento,
    estimaciones)
320 - Equipo de QA (estrategia de testing, bugs,
    releases)
321 - DevOps/Infraestructura (deployments, incidentes
    , ambientes)
322 - Otros Team Leads (si existieran en el futuro)
323
324 ### Stakeholders:
325 - CTO (indirectamente, a través del Gerente)
326 - Áreas de negocio (Comercial, Operaciones) en
    proyectos específicos
327
328 ---
329
330 ## ESCENARIOS PRÁCTICOS DE DECISIÓN
331
332 Para clarificar el ámbito de decisión, se
    presentan ejemplos concretos de situaciones
    comunes:
333
334 ### Escenario 1: Selección de Librería para Manejo
    de Fechas
335
336 **Situación:** El equipo necesita una librería
    para manejo de fechas en el frontend.
337
338 **Opciones:** date-fns, moment.js, day.js, luxon
```

339
340 ****¿Quién decide?****
341 - ****Team Lead decide autónomamente**** si:
342 - La librería es de uso aislado en pocos
componentes
343 - No hay estándar definido previamente por el
Gerente/Arquitectura
344 - El costo es <\$500 (la mayoría son gratuitas)
345
346 - **▲** ****Escalar al Gerente**** si:
347 - La decisión afecta a múltiples equipos o
establece un estándar
348 - Hay implicaciones de performance
significativas
349 - Requiere cambiar una decisión arquitectónica
previa
350
351 ****En este caso:**** Team Lead decide autónomamente,
documenta en ADR interno del equipo
352
353 ---
354
355 **### Escenario 2: Developer con Bajo Desempeño**
356
357 ****Situación:**** Un developer junior lleva 3 semanas
sin cumplir con las tareas del sprint de manera
consistente.
358
359 ****¿Quién decide?****
360 - ****Team Lead maneja autónomamente**** si:
361 - Es situación reciente (<2 meses)
362 - Puede abordarse con coaching y feedback
363 - No hay riesgo de compromisos críticos
364
365 ****Acciones del Team Lead:****
366 1. 1:1 para entender causas raíz
367 2. Plan de mejora con objetivos claros
368 3. Seguimiento semanal por 1 mes
369 4. ****Informar al Gerente**** de la situación y plan
370
371 - **▲** ****Escalar al Gerente**** si:

```
372 - El problema persiste >2 meses
373 - Hay impacto en compromisos con clientes
374 - Se requiere acción formal (PIP, cambio de rol
    , desvinculación)
375
376 ---
377
378 ### Escenario 3: Cambio en la Arquitectura de un
    Módulo
379
380 **Situación:** El equipo identifica que un módulo
    debería migrar de arquitectura monolítica a
    microservicio.
381
382 **¿Quién decide?**
383 - **NO decide Team Lead autónomamente**
384 - **Debe escalar al Gerente** porque:
385 - Es decisión arquitectónica que afecta
    múltiples sistemas
386 - Requiere >2 sprints de esfuerzo
387 - Puede generar dependencias con otros equipos
388 - Requiere validación de Arquitectura
389
390 **Rol del Team Lead:**
391 1. Documentar el problema y propuesta técnica
392 2. Presentar al Gerente con análisis de costo-
    beneficio
393 3. Gerente coordina con Arquitectura
394 4. Team Lead implementa una vez aprobado
395
396 ---
397
398 ### Escenario 4: Herramienta de Productividad para
    el Equipo
399
400 **Situación:** El equipo quiere adoptar una
    herramienta de pair programming remoto (
    CodeTogether, Live Share Plus).
401
402 **¿Quién decide?**
403 - **Team Lead decide autónomamente** si:
```

404 - Costo <\$50/mes por persona (total <\$500/mes)
405 - Es para uso del equipo específico
406 - No interfiere con políticas de seguridad
407
408 - ▲ **Escalar al Gerente** si:
409 - Costo >\$500/mes total
410 - Requiere cambios en políticas o aprobación de
IT/Seguridad
411 - Se busca estandarizar en toda el área
412
413 **En este caso:** Team Lead aprueba trial gratuito
, evalúa con el equipo, decide adopción si <\$500/
mes
414
415 ---
416
417 ### Escenario 5: Re-priorización de Tareas en el
Sprint
418
419 **Situación:** A mitad del sprint surge un bug
crítico en producción que requiere atención
inmediata.
420
421 **¿Quién decide?**
422 - **Team Lead decide autónomamente** si:
423 - Puede resolverse ajustando prioridades dentro
del sprint
424 - No afecta compromisos críticos del sprint con
clientes
425 - La capacity lo permite sin poner en riesgo el
objetivo del sprint
426
427 **Acciones del Team Lead:**
428 1. Evaluar severidad e impacto del bug
429 2. Re-priorizar tareas del sprint
430 3. Asignar recurso al bug crítico
431 4. **Comunicar al Gerente** el ajuste y riesgos
432
433 - ▲ **Escalar al Gerente** si:
434 - El bug requiere cancelar features
comprometidas

435 - Hay necesidad de recursos adicionales
436 - Afecta compromisos con clientes externos
437
438 ---
439
440 ### Escenario 6: Developer Solicita Curso de \$1,500
441
442 **Situación:** Un developer senior quiere asistir a un curso avanzado de arquitectura de \$1,500.
443
444 **¿Quién decide?**
445 - **NO decide Team Lead autónomamente**
446 - **Debe escalar al Gerente** porque:
447 - Involucra presupuesto de capacitación
448 - Excede autoridad financiera del Team Lead
449 - Requiere aprobación presupuestaria
450
451 **Rol del Team Lead:**
452 1. Evaluar relevancia del curso para el rol del developer
453 2. Preparar justificación de ROI
454 3. Presentar recomendación al Gerente
455 4. Gerente decide basándose en presupuesto disponible
456
457 ---
458
459 ### Escenario 7: Adopción de TypeScript en Proyecto React
460
461 **Situación:** El equipo propone migrar un proyecto React de JavaScript a TypeScript.
462
463 **¿Quién decide?**
464 - ▲ **Depende del alcance:**
465
466 Si es **proyecto pequeño, aislado** (<5 componentes, <1 sprint):
467 - Team Lead puede decidir si no hay estándar contrario

```
468
469 Si es **proyecto significativo** (>1 sprint,
    múltiples módulos):
470 - **Debe escalar al Gerente** porque:
471   - Es decisión técnica de alto impacto
472   - Requiere validación con Arquitectura
473   - Puede establecer precedente para otros
    proyectos
474   - Necesita plan de migración y capacitación
475
476 ---
477
478 ### Escenario 8: Refactoring de Código Legacy
479
480 **Situación:** El equipo identifica código legacy
    que dificulta el desarrollo y propone
    refactorizarlo.
481
482 **¿Quién decide?**
483 - **Team Lead decide autónomamente** si:
484   - Es refactoring interno sin cambiar interfaces
    públicas
485   - Se puede completar en 1-2 sprints
486   - No afecta otros equipos o sistemas
487   - Es parte de la deuda técnica operativa
488
489 **Acciones del Team Lead:**
490 1. Priorizar dentro de la capacity de deuda
    técnica del sprint (típicamente 10-20%)
491 2. Asegurar cobertura de tests antes y después
492 3. Comunicar al Gerente en 1:1 semanal
493
494 - ▲ **Escalar al Gerente** si:
495   - Requiere >2 sprints o >40 horas
496   - Cambia contratos públicos o APIs
497   - Afecta arquitectura o integración con otros
    sistemas
498   - Es deuda técnica arquitectónica (no operativa)
499
500 ---
501
```

502 ## DIFERENCIACIÓN CON OTROS ROLES

503

504 ### VS. GERENTE DE DESARROLLO:

505

506 - ****Foco:**** Operativo/Táctico (día a día) vs
Estratégico (mediano/largo plazo)

507 - ****Alcance:**** Ejecución de proyectos vs
Definición de estrategia y roadmap

508 - ****Gestión:**** Desarrollo técnico y coaching vs
Gestión de carrera, presupuesto y estructura

509 - ****Decisiones:**** Técnicas de implementación vs
Arquitectónicas y de negocio

510

511 ### VS. SENIOR DEVELOPER:

512

513 ****Comparación Detallada:****

514

515 | Dimensión | Team Lead de Desarrollo | Senior
Developer |

516 |-----|-----|-----
-----|

517 | ****Responsabilidad Principal**** | Resultado y
éxito del equipo completo (14 personas) |
Contribución técnica individual excepcional |

518 | ****Foco de Trabajo**** | 50% Liderazgo/Gestión + 50
% Técnico | 80-90% Técnico + 10-20% Mentoría
informal |

519 | ****Distribución de Tiempo**** | Code reviews, 1:1s
, coordinación, desarrollo | Desarrollo hands-on,
diseño técnico, investigación |

520 | ****Accountability**** | Cumplimiento de sprints,
calidad del equipo, desarrollo de personas |
Entrega de features complejas, calidad técnica,
expertise |

521 | ****Impacto**** | Multiplicador (a través del equipo
) | Directo (features críticas, arquitectura) |

522 | ****Decisiones**** | Asignación de tareas,
priorización, procesos del equipo | Decisiones de
implementación individual |

523 | ****Reuniones**** | Facilita ceremonias, 1:1s con 14
personas, coordinación interfuncional | Participa

523 en planning/retros, refinamiento técnico |
524 | ****Gestión de Personas**** | Formal: evaluaciones,
feedback, planes de desarrollo | Informal:
mentoría, pair programming, code reviews |
525 | ****Visión**** | Éxito del equipo a nivel sprint y
trimestre | Excelencia técnica de features
específicas |
526
527 ****Indicadores Clave de Diferenciación:****
528
529 - ****Senior Developer**** que quiere evolucionar a ****Team Lead**** debe demostrar:
530 - Capacidad de multiplicar impacto a través de
otros
531 - Interés genuino en desarrollo de personas
532 - Habilidad de delegar y no solo hacer todo él/
ella
533 - Visión de equipo por sobre contribución
individual
534 - Disposición a reducir tiempo de código por
liderazgo
535
536 - ****Team Lead**** que quiere volver a ****Senior Developer**** (track técnico puro):
537 - Prefiere impacto técnico directo sobre gestión
de personas
538 - Busca profundizar expertise técnico vs.
ampliar liderazgo
539 - Mayor interés en arquitectura/diseño que en
operación de equipos
540
541 ---
542
543 **## CONDICIONES DEL ROL**
544
545 - ****Modalidad de Trabajo:**** Híbrida/Remota (según
políticas de la empresa)
546 - ****Disponibilidad:**** Flexible para cubrir
necesidades del equipo (reuniones, blockers,
incidentes)
547 - ****Participación en Guardias:**** Puede participar

547 en rotación de guardias on-call según necesidad
548 - ****Código Activo:**** Se espera que dedique 30-50%
de su tiempo a desarrollo hands-on
549
550 ---
551
552 **## PROGRESIÓN DE CARRERA**
553
554 Este rol puede evolucionar hacia:
555 - ****Gerente de Desarrollo de Software**** (con
demostración de habilidades de gestión estratégica
y de negocio)
556 - ****Engineering Manager**** (en organizaciones con
estructura dual IC/Management)
557 - ****Arquitecto de Software**** (con foco más
profundo en arquitectura y menos en gestión de
personas)
558 - ****Team Lead de equipos más grandes o de mayor
complejidad****
559
560 ---
561
562 **## FECHA DE CREACIÓN Y REVISIONES**
563
564 Documento creado: Noviembre 2025
565 Última revisión: Noviembre 2025 - Alineación con
rol de Gerente de Desarrollo y clarificación de
ámbito de decisión
566 Próxima revisión: Noviembre 2026
567
568 ---
569
570 **## APROBACIONES**
571
572 - ****Creado por:**** [CTO/Gerente de Desarrollo]
573 - ****Aprobado por:**** [CTO]
574 - ****Fecha de aprobación:**** [Fecha]
575
576 ---
577
578 ***Este documento es confidencial y de uso interno**

578 de Goodline/Maxirest. La descripción de rol puede
ser actualizada según las necesidades de la
organización.*

579